Weighted tuples to packet rows: a source-refinement audit

5 November 2026. PCC-20261105-c0901e — unverified statement. This packet supplies a concrete state contract, transition-by-transition review argument and source correspondence. No Lean build, Rust refinement proof, new algebraic identity or general complexity theorem is claimed. All audit arguments below are unverified producer assertions. [pcc:unverified]

The component advance is the explicit connection between the tuple driver's target usage and a child packet's incoming degree. The review argument gives q <= E <= R for driver-generated packets under the listed preconditions: q positive incoming records, total incoming degree E, R selected rows. It also identifies why the unchecked integer updates can fit under that contract. These interfaces are now stated rather than left as an unnamed link in the previous cost map (../hashed_ecdl_entry_cost_20261104/README.md). [pcc:unverified]

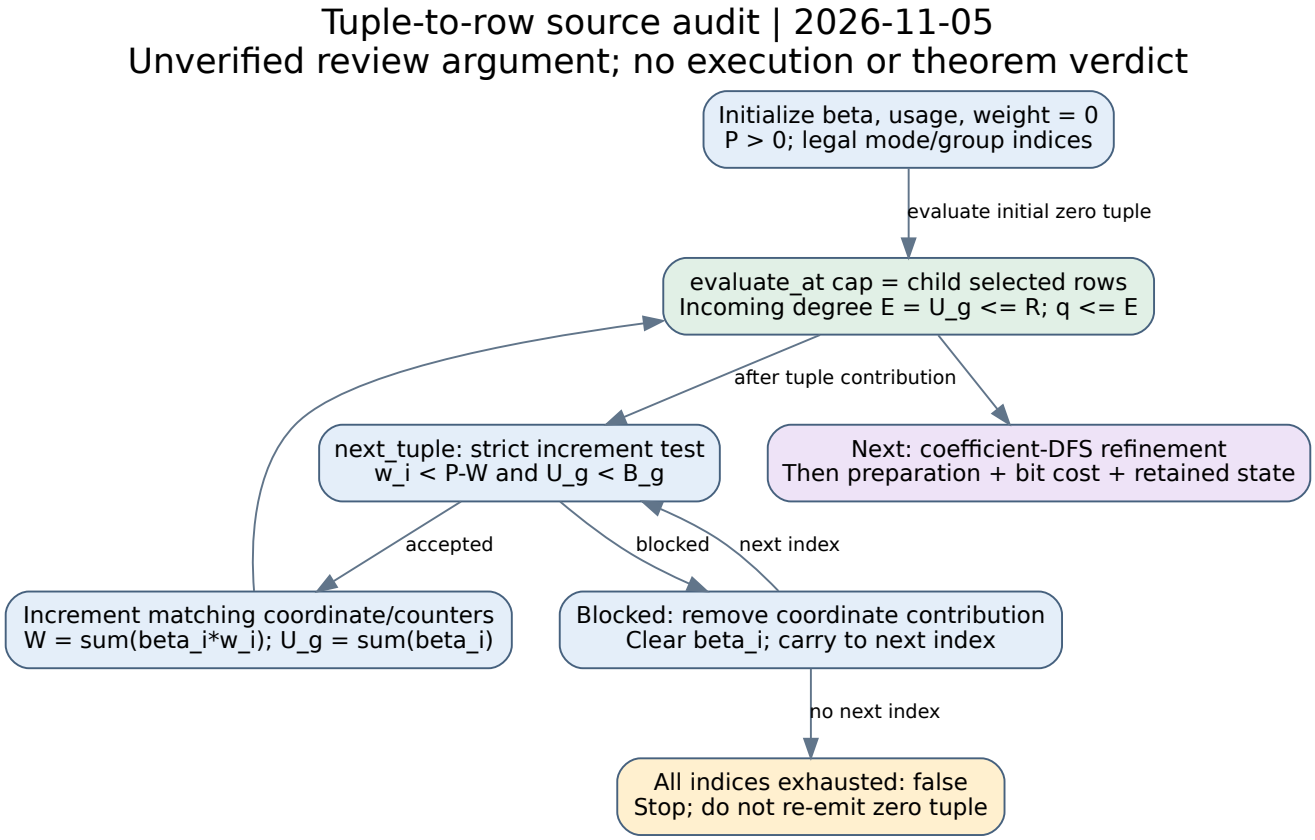


Figure 1: Evidence dependencies; no timing data.

Sources and boundaries

Native source is pinned to crypto f96a6d475896e06065eda55ccecf5f0132692423. Relevant contraction blobs are unchanged from the Nov4 source map. sources.json (sources.json) pins the source and prior reports; audit-contract.json (audit-contract.json) records the predicates, transitions, source sites and unproved bridges. [pcc:unverified]

Read OpenAI Appendix D.1 and D.4.5-D.4.7, from the publisher's current PDF path at last-change commit 42907c9e999cc00d6abe7a50bfc1023250edd1b9, PDF Git blob 0503566e95bc587a4076573a45132d97531b8ba6. D.4.5 describes weighted tuples and zero continuations; D.4.6 defines packet/group coefficient extraction and creation-time descriptions. D.1 permits an exponential entry-work allowance. This review addresses the local enumerator/row interface, with the full semantic and resource induction still open. [pcc:unverified]

Current atlas matching at crypto-autoresearcher 8fcfa58fee923c48f96ecf4126ffe4b1ea5596e7 returns the existing LEAD-EXO-hashed-factor-base-tensor-ecdl. Its independent-audit action owns this work. Related black-box cost-model rows remain scoped to their own assumptions. Stale native-test/mirroring descriptions in that lead are not promoted to current evidence. No full atlas-delta stamp, novelty claim or new mechanism is made. [pcc:unverified]

Contract and caller provenance

Let n be the number of constructed modes, P the current evaluation ring's positive precision, r the radix and K the number of packets. For mode i, write w_i for its weight and g_i = packet_i*r + target_i. Let beta_i be its exponent, W the driver's weight, U_g its usage counters and B_g its row caps. These symbols describe one call, not a root-wide precision convention.

The proposed invariant at every published tuple is:

1. beta has n entries; U and B have K*r entries; each g_i is in range.

2. Each w_i is positive. W equals the sum of beta_i*w_i and W < P.

3. For each group key g, U_g equals the sum of beta_i with g_i = g, and U_g <= B_g.

evaluate_at initializes beta, U and W to zero. Ring::new and at_precision reject zero precision; modes assigns weight b+1 and records a packet index and target digit. The K*r vector length uses checked multiplication. A valid graph/packing, aligned label arrays, positive radix, legal digit coordinate and valid mode records are caller preconditions; this packet does not formalize packing or the label/digit construction. [pcc:unverified]

Transition audit

Site	Exact source action	Review argument / obligation
Initialization	Zero beta, usage and weight; evaluate before advancing	Zero satisfies the sums and caps when P>0; zero tuple is included
Increment test	w_i < P-W AND U_g < B_g	Strict weighted cutoff is preserved; equality with P is rejected
Accepted increment	beta_i, W and U_g each receive their matching increment	Both sums change by precisely the incremented coordinate's contribution
Rejected coordinate	Subtract beta_i*w_i and beta_i from its two counters, then clear beta_i	Restores the sums for the residual tuple; lower indices already reset stay zero
Carry	Try the next mode only after the reset	Bounds are checked against the updated residual state, including modes sharing a group
Successful return	Return immediately at the first feasible increment	Lower indices are zero; higher indices are unchanged
Exhaustion	All coordinates reset; return false	Caller stops and does not emit the restored zero tuple again
Meter error	Charge precedes a coordinate's mutation and propagates with ?	Completed resets remain consistent; interrupted enumeration returns an error, not a partial total

The reset proof uses nonnegative summands: beta_i*w_i is at most W, and beta_i is at most its group's usage. Thus subtraction removes an actual contribution. The increment test is exactly the source's strict inequality; replacing it by a non-strict test would change the enumerated domain. [pcc:unverified]

Coverage and termination review argument

Order exponent vectors with coordinate0 least significant. For fixed higher coordinates the constraints are downward closed in the remaining coordinates. Every feasible partial vector therefore has an all-zero completion. An increment cannot jump over a feasible value of its coordinate; when an increment is blocked, larger values with the same higher coordinates are also blocked. Clearing that coordinate and carrying reaches the next possible higher-coordinate vector. These are the steps a refinement proof must join to obtain exact-once coverage of the constructed modes' feasible tuples. [pcc:unverified]

Positive weights and W<P bound each exponent by P-1. Every successful call increases the least-significant-first ordering on that finite domain. This gives the proposed termination argument; it does not bound cost by elapsed time or prove the manuscript's total child count. Equal-looking mode records remain separate indexed variables: no coefficient or mode is deduplicated. [pcc:unverified]

The empty-mode case evaluates its unique empty tuple once, then returns false without indexing a group. At P=1, positive weights admit only zero. A group with B_g=0 cannot receive a positive exponent. At terminal false, the caller exits rather than reusing the zeroed state. A meter limit aborts the computation through Result; it does not produce a complete-count receipt. [pcc:unverified]

Machine-integer obligations

The source updates use usize rather than checked operations at every site. The review argument therefore needs the natural-number equalities above, not just informal similarity between the counters.

Expression	Condition sufficient for representability
packet*r + target	packet<K, target<r, and successful checked K*r allocation length
P-W	0<=W<P, including residual states during carry
beta_i+1	beta_i<=U_g<B_g<=usize::MAX at an accepted increment
W+w_i	w_i<P-W, so the result is strictly below representable P
U_g+1	U_g<B_g, so the result is at most representable B_g
beta_i*w_i and reset subtraction	This summand is at most W; both removals are of nonnegative contributions
Bound-cell increment	A cell counts at most the packet's real slot occurrences
Incoming exponent sum	Its partial sums are at most E<=R, a representable selected-row count

This is a conditional arithmetic audit, not a claim that arbitrary inputs to the private helper are checked. Corrupted slices/counters, zero precision, invalid group indices or malformed packing lie outside the contract and need their own caller/refinement proof. Error and admission guards are unchanged. [pcc:unverified]

Join the counters to child rows

For packet p and group g, evaluate_at counts the real slot occurrences whose left endpoint's row-label digit is g. child fills row_map using that same digit predicate, then selects packet slots whose left endpoint has a row_map entry. Thus its selected row list has B_(p,g) entries, provided the shared packing/label preconditions hold. This equality concerns row slots; the column-side group predicate is separate. [pcc:unverified]

child visits every paired mode/exponent with zip, checks equal vector lengths, and retains precisely the positive records with the current packet and target group as Incoming. It does not deduplicate records. Consequently the proposed join is E=sum(beta_i for g_i=(p,g))=U_(p,g)<=B_(p,g)=R. Each retained exponent is positive, giving q<=E. Sparse missing row coefficients are filled with zero; they do not change the row list or these exponent sums. Outgoing records are selected by source, and are not bounded by this target's R. [pcc:unverified]

This supplies a concrete interface to check in PacketTensor::entry: its incoming remaining sum equals E and fits the row count for packets created by this driver. Under the contract, the initial E>R shortcut is unreachable on that path. Keep the guard intact: arbitrary packet descriptions and the completeness of the source-to-packet bridge have not been verified. The earlier preparation-before-shortcut observation remains true. [pcc:unverified]

To transfer the extra caps to D.16, the remaining semantic obligation is explicit: the target row product is affine in the formal mode variables, with at most R factors, so overfull target degree has zero coefficient. One must then show the native coefficient DFS extracts that coefficient with signs and all contributions. The state audit alone is not a proof of that semantic correspondence or the recursion identity. [pcc:unverified]

What this advances, and the next discriminating packet

The candidate q<=E<=R interface eliminates q as an unconstrained independent parameter for driver-created entries. It ties the incoming coefficient lists to the selected rows and gives a row-count condition for the remaining-degree sum. The DFS still explores choices at each row, including rejected choices; preparation still builds local R-by-C data and incoming coefficient lists. No traversal, allocation or arithmetic work is omitted by this audit. [pcc:unverified]

Next, instantiate a faithful source refinement of this contract and audit the coefficient DFS's remaining-exponent/ restoration invariant against the signed affine row product. Then combine preparation, outgoing powers and their operand widths with all ancestor descriptions, modes, samples and jets. Creation precision, strict shrinkage and the manuscript's precision bounds remain required for the aggregate bit/retention induction.

Lean/lake are absent in this environment; no theorem module or transitive kernel/axiom check ran. No polynomial identity certificate was issued and no new algebraic identity is asserted. A formal model relating Rust slices, usize updates and packing predicates is still missing. The general-audit problem remains open; this deliverable is its local source-audit packet. [pcc:unverified]

Evidence and delivery

The existing three-mode enumeration unit test was read as source; it was not rerun and is not evidence for arbitrary instances. No new scientific experiment, benchmark, timing, challenge solve or cloud job ran. All previous reports/ manifests, runtime code and dependencies are preserved.

The validation note (VALIDATION.md), PDF (report.pdf), editable diagram (dependency.dot) and renderer (render_report.py) accompany this packet. Canonical leaderboard, progress, curve and performance graphs receive no new value or transfer edge.